02_score_models

```r
source("R/00_setup.R")
```

```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```r
library(jsonlite)
```

```

Attaching package: 'jsonlite'

The following object is masked from 'package:purrr':

    flatten
```

```r
library(haven)

#' ## Stage 1 - Read the elicitation panel
#'
#' One JSONL per model × country. Only records flagged usable are kept: `OK`,
#' `NORMALIZED` (probabilities rescaled to sum to one), and `SUM_SOFT_OVER`
#' (sum slightly above one, within tolerance). Provider and model are read from
```

```r
#' inside each record rather than from the filename. Output is long - one row per
#' response option - for clean downstream joins.

USABLE <- c("OK", "NORMALIZED", "SUM_SOFT_OVER")

read_one <- function(path) {
  lines <- readLines(path, warn = FALSE); lines <- lines[nzchar(lines)]
  recs  <- lapply(lines, fromJSON, simplifyVector = TRUE)
  map_dfr(recs, function(r) {
    if (is.null(r$flag) || !(r$flag %in% USABLE)) return(NULL)
    p <- r$parsed
    if (is.null(p) || !length(p)) return(NULL)
    tibble(provider = r$provider, model = r$model,
           country  = r$country,  condition = r$condition,
           item_id  = r$item_id,  item_type = r$item_type,
           wave     = as.integer(r$wave),
           replicate = as.integer(r$replicate),
           option   = names(p),
           prob     = as.numeric(unlist(p)))
  })
}

files <- list.files(ELICITATION_DIR, pattern = "\\.jsonl$", recursive = TRUE, full.names = TI
cat("reading", length(files), "files...\n")
```

```
reading 160 files...
```

```r
raw <- map_dfr(files, read_one)

#' Collapse replicates to the mean stated distribution per cell and option. The
#' per-option SD across replicates is retained as a precision signal - it is the
#' within-cell variability that the country bootstrap does not separately
#' propagate, and it is largest for the least stable model.

cell_keys <- c("provider", "model", "country", "condition", "item_id", "item_type", "wave")

repl_n <- raw %>% distinct(across(all_of(cell_keys)), replicate) %>%
  count(across(all_of(cell_keys)), name = "k_used")

model_cells <- raw %>%
  group_by(across(all_of(c(cell_keys, "option")))) %>%
```

```r
  summarise(mean_prob = mean(prob), sd_prob = sd(prob), .groups = "drop") %>%
  left_join(repl_n, by = cell_keys)

saveRDS(model_cells, "model_cells_long.rds")

#' Completeness gate before scoring: four models, forty countries each, and
#' k_used = 10 for every cell.

cat("\nrows:", nrow(model_cells), "\n")
```

```
rows: 58188
```

```r
model_cells %>% distinct(model, country) %>% count(model, name = "countries") %>% print()
```

```
# A tibble: 4 x 2
  model              countries
  <chr>                  <int>
1 Qwen/Qwen3.6-Plus         40
2 claude-opus-4-8           40
3 gemini-3.6-flash          40
4 gpt-5.5                   40
```

```r
model_cells %>% distinct(across(all_of(cell_keys)), k_used) %>%
  count(model, k_used) %>% arrange(model, k_used) %>% print(n = Inf)
```

```
# A tibble: 4 x 3
  model              k_used     n
  <chr>               <int> <int>
1 Qwen/Qwen3.6-Plus      10  2100
2 claude-opus-4-8        10  2100
3 gemini-3.6-flash       10  2100
4 gpt-5.5                10  2100
```

```r
#' ## Stage 2 - Stated distributions to WVS component values
#'
#' Three item types, each converted by the rule that makes it comparable to the
#' corresponding survey respondent mean.
#'
```

```r
#' *Ordinal items* become the expected value of the stated distribution over the
#' numbered scale.

model_cells <- readRDS("model_cells_long.rds")

ordinal_ids <- c("F118", "F120", "F063", "G006", "E018", "A008", "E025", "A165")
ev_A <- model_cells %>%
  filter(item_type == "A", item_id %in% ordinal_ids) %>%
  mutate(pt = as.numeric(option)) %>%
  group_by(model, country, condition, wave, item = item_id) %>%
  summarise(value = sum(pt * mean_prob), .groups = "drop")

#' *The child-qualities card sort* yields the four child-autonomy components as
#' stated selection probabilities. The deck differs across waves, so each quality
#' is mapped to its wave-specific card position.

scored_key <- tribble(
  ~wave, ~card, ~item,
  4L,  1L, "A029",  4L,  7L, "A039",  4L,  8L, "A040",  4L, 10L, "A042",
  5L,  1L, "A029",  5L,  7L, "A039",  5L,  8L, "A040",  5L, 10L, "A042",
  6L,  1L, "A029",  6L,  7L, "A039",  6L,  8L, "A040",  6L, 10L, "A042",
  7L,  2L, "A029",  7L,  8L, "A039",  7L,  9L, "A040",  7L, 11L, "A042"
)
ev_B <- model_cells %>%
  filter(item_type == "B") %>%
  mutate(card = as.integer(option)) %>%
  inner_join(scored_key, by = c("wave", "card")) %>%
  transmute(model, country, condition, wave, item, value = mean_prob / 100)

#' *The four-goal post-materialism ranking* becomes Y002 by assigning each
#' ordered goal-pair the number of post-materialist goals it contains (goals 2
#' and 4) and taking the probability-weighted mean on the resulting 1-3 index.

y002_cat <- function(pair) { g <- as.integer(strsplit(pair, ",")[[1]]); 1 + sum(g %in% c(2L,
pair_cat <- model_cells %>% filter(item_id == "Y002") %>% distinct(option) %>%
  mutate(cat = map_dbl(option, y002_cat))
ev_C <- model_cells %>%
  filter(item_id == "Y002") %>%
  left_join(pair_cat, by = "option") %>%
  group_by(model, country, condition, wave, item = item_id) %>%
  summarise(value = sum(mean_prob * cat), .groups = "drop")
```

```r
model_item_values <- bind_rows(ev_A, ev_B, ev_C) %>%
  arrange(model, country, condition, wave, item)
saveRDS(model_item_values, "model_item_values.rds")

#' Expect thirteen values per cell, and twelve for C0 - national pride (G006) is
#' undefined without a country and is omitted from the country-agnostic
#' condition. That difference in indicator base is a caveat on any C0-to-C1
#' comparison; see script 05.

model_item_values %>% count(model, country, condition, wave) %>%
  count(n_items = n) %>% print()
```

```
# A tibble: 2 x 2
  n_items     n
    <int> <int>
1      12   160
2      13   696
```

```r
#' Scale sanity check only, not a level result: model item means should sit on
#' the same scale as the WVS means.

params <- read_csv("standardization_params.csv", show_col_types = FALSE)
model_item_values %>% group_by(item) %>%
  summarise(model_mean = mean(value), .groups = "drop") %>%
  left_join(params %>% select(item, wvs_mu = mu), by = "item") %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>% print(n = Inf)
```

```
# A tibble: 13 x 3
   item  model_mean wvs_mu
   <chr>      <dbl>  <dbl>
 1 A008       2.01   1.89
 2 A029       0.345  0.497
 3 A039       0.321  0.365
 4 A040       0.312  0.41
 5 A042       0.313  0.379
 6 A165       1.70   1.72
 7 E018       1.48   1.55
 8 E025       1.94   2.20
 9 F063       7.25   7.67
10 F118       4.32   3.58
11 F120       4.14   3.23
```

```
12 G006       1.53   1.55
13 Y002       1.80   1.78
```

```r
#' ## Stage 3 - The per-country indicator base
#'
#' Reproduces the ground truth's keep/drop table so the model averages over the
#' same item set per country. Drops are not random: they concentrate in items
#' thin in less-liberal and less-Western contexts, and fall more heavily on
#' Dimension 2. Nine countries rest on a reduced base and are correspondingly
#' less precisely estimated.

sub <- readRDS("WVS_subset_40countries_W4toW7.rds")
resolve <- function(want, have) unname(setNames(have, toupper(have))[toupper(want)])
nms <- names(sub); item_cols <- resolve(ITEMS, nms); stopifnot(!any(is.na(item_cols)))

X <- vapply(item_cols, function(cc) { v <- as.numeric(zap_labels(sub[[cc]])); v[v < 0] <- NA
            numeric(nrow(sub)))
colnames(X) <- ITEMS
wave <- as.numeric(sub[[resolve("S002VS", nms)]]); iso <- sub$iso3

item_base <- data.frame(iso, wave, X, check.names = FALSE) %>%
  group_by(iso, wave) %>%
  summarise(across(all_of(ITEMS), ~ mean(!is.na(.x)) > 0.5), .groups = "drop") %>%
  group_by(iso) %>%
  summarise(across(all_of(ITEMS), ~ all(.x)), .groups = "drop") %>%
  pivot_longer(all_of(ITEMS), names_to = "item", values_to = "keep")

write_csv(item_base, "item_base_by_country.csv")

cat("countries:", n_distinct(item_base$iso), "| rows:", nrow(item_base), "(expect 520)\n")
```

```
countries: 40 | rows: 520 (expect 520)
```

```r
item_base %>% group_by(iso) %>% summarise(n_kept = sum(keep), .groups = "drop") %>%
  count(n_kept) %>% arrange(desc(n_kept)) %>% print()
```

```
# A tibble: 5 x 2
  n_kept     n
   <int> <int>
1     13    31
2     12     3
```

```
3      11      2
4      10      3
5       8      1
```

```r
item_base %>% filter(!keep) %>% arrange(iso, item) %>% print(n = Inf)
```

```
# A tibble: 21 x 3
   iso   item  keep
   <chr> <chr> <lgl>
 1 CHN   A040  FALSE
 2 CHN   E025  FALSE
 3 CHN   F063  FALSE
 4 EGY   F063  FALSE
 5 EGY   F118  FALSE
 6 EGY   F120  FALSE
 7 HKG   E025  FALSE
 8 HKG   Y002  FALSE
 9 IRN   E025  FALSE
10 IRQ   A039  FALSE
11 IRQ   E018  FALSE
12 IRQ   E025  FALSE
13 IRQ   F063  FALSE
14 IRQ   F118  FALSE
15 MAR   F118  FALSE
16 PER   F118  FALSE
17 PER   F120  FALSE
18 PER   G006  FALSE
19 SGP   E025  FALSE
20 TUR   F118  FALSE
21 TUR   F120  FALSE
```

```r
#' ## Stage 4 - Project onto the axes
#'
#' Standardize each model item value with the *survey's*   and  , apply the
#' orientation sign, and average within each dimension over only that country's
#' kept items. Identical rule to the ground truth, so the output is directly
#' comparable to `country_wave_coords.rds`.

vals   <- readRDS("model_item_values.rds")
params <- read_csv("standardization_params.csv", show_col_types = FALSE)
base   <- read_csv("item_base_by_country.csv",   show_col_types = FALSE)
```

```r
model_coords <- vals %>%
  left_join(params, by = "item") %>%
  left_join(base,   by = c("country" = "iso", "item")) %>%
  filter(keep) %>%
  mutate(z = sign * (value - mu) / sd) %>%
  group_by(model, country, condition, wave, dim) %>%
  summarise(score = mean(z), n_items = n(), .groups = "drop") %>%
  pivot_wider(names_from = dim, values_from = c(score, n_items)) %>%
  rename(Dim1 = score_Dim1, Dim2 = score_Dim2,
         k_dim1 = n_items_Dim1, k_dim2 = n_items_Dim2)

saveRDS(model_coords, "model_coords.rds")

cat("rows:", nrow(model_coords), "| models:", n_distinct(model_coords$model),
    "| countries:", n_distinct(model_coords$country), "\n")
```

```
rows: 856 | models: 4 | countries: 40
```

```r
model_coords %>% count(condition) %>% print()
```

```
# A tibble: 3 x 2
  condition     n
  <chr>     <int>
1 C0          160
2 C1          160
3 C2          536
```

```r
#' Model coordinates should occupy a range roughly comparable to the ground
#' truth: same scale, centred near zero, no blow-up.

model_coords %>% summarise(across(c(Dim1, Dim2), list(min = min, mean = mean, max = max))) %>
  mutate(across(everything(), ~ round(.x, 3))) %>% print()
```

```
# A tibble: 1 x 6
  Dim1_min Dim1_mean Dim1_max Dim2_min Dim2_mean Dim2_max
     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>    <dbl>
1   -0.524     0.031    0.696   -0.522     0.092    0.851
```

```r
readRDS("country_wave_coords.rds") %>%
  summarise(across(c(Dim1, Dim2), list(min = min, mean = mean, max = max))) %>%
  mutate(across(everything(), ~ round(.x, 3))) %>% print()
```

```
# A tibble: 1 x 6
  Dim1_min Dim1_mean Dim1_max Dim2_min Dim2_mean Dim2_max
     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>    <dbl>
1    -0.65    -0.001    0.831   -0.842    -0.008    0.753
```